Heat Production Optimization

Semester 2, Year 2026

Students: David Avramov daavr25@student.sdu.dk
Gintaras Zulys gizul25@student.sdu.dk
Leon Jürgen Thye lethy25@student.sdu.dk
Nadia Kristensen nkris25@student.sdu.dk
Oliwia Roksana Moskalik olmos25@student.sdu.dk
Yaroslav Savenko yasav25@student.sdu.dk

Supervisor: Alan Sieradzki

Lecturers: Adam Alami
Amir Ghomashi

University of Southern Denmark
May 22, 2026

Contents

List of Figures

List of Tables

Abstract

Efficient heat production and energy management are becoming increasingly important in modern district heating systems, particularly in situations where operational costs, electricity prices, and energy demand continuously fluctuate. The project focused on developing a software solution capable of optimizing heat production schedules while ensuring stable heat delivery to consumers and improving overall economic performance. The system analyses production unit configurations, heat demand data, and electricity market conditions to determine the most cost-effective production strategy across different operational scenarios.

The work was carried out using Scrum methodology and iterative software development practices. Multiple sprints were used to organise tasks, manage collaboration, and continuously improve the system through planning sessions, reviews, and retrospectives. Alongside the project management aspects, significant attention was given to software architecture, system modelling, implementation, testing, and code quality.

The document presents the complete development process, beginning with the project background, objectives, and requirements, followed by the Scrum implementation and sprint activities. It also explains the technical structure of the solution through UML diagrams, architectural descriptions, frontend and backend implementation details, and data management strategies. In addition, reflections on teamwork, testing, refactoring, code reviews, and requirements engineering are discussed to evaluate both the technical and collaborative outcomes of the project.

By exploring the different stages of development and the decisions made throughout the process, the reader gains insight into how Agile methodologies and software engineering practices can be applied to create a functional optimization system for district heating production.

Chapter 1

Introduction

1.1 Project Objectives and Requirements

The goal of the project is to define heat schedules for all available production units with the lowest possible expenses and the highest profit on the electricity market. For the development, the Scrum framework was adopted. Communication between the team and product owners was established early on to provide a better feedback loop and an opportunity for thorough requirement elicitation, which were compiled in a table (Appendix E), and those prescribed in two scenarios. The user should be able to see visual representation of all relevant data, assets, optimization results, as well as being able to configure each individual asset, optimizer itself and export the results in a convenient format. The system shall ensure that heat demand is met at any time, calculate net cost for each production unit and prioritize each unit based on it, create separate result data for each production unit and calculate the optimized heat schedule.

1.2 Problem Statement

HeatItOn utility company is supplying heat to a single district heating network with nearly two thousand buildings. They utilize different units for heat production, such as traditional gas boilers or units that either consume or produce electricity. At the moment, the company does all its scheduling of units manually. However, manual scheduling is both time consuming and prone to human error. Managing all the units, constantly following the electricity prices that change hourly, and ensuring that the heat demand is met for all the buildings on the grid makes it harder to optimize the

production for the best profit. Moreover, the time spent on this task could be better utilized on other, higher-value activities.

An automated system for optimizing the production for profit can significantly speed up the scheduling of units and save human resources otherwise spent on manual calculations. This would ensure that the heat production is as profitable as it can be and improve accuracy by reducing human error. The project aims to develop a heat optimizer software that can take in all the units and their data and calculate a unit schedule that will be optimized for the highest profit. It takes into consideration the hourly shifting electricity prices and the different heating seasons to make the best and most profitable choices for the unit schedule.

Chapter 2

Scrum Implementation

The project was developed using the Scrum framework to support iterative development, teamwork, and continuous progress throughout the implementation of the heat production optimization system. The work was divided into six sprints, with a different Scrum Master assigned for each sprint. This rotation was a team decision made to ensure that every member gained experience with sprint coordination and Agile project management.

The Scrum Master was responsible for coordinating meetings, preparing agendas for supervisor meetings, writing meeting logs, and ensuring that Scrum practices were followed throughout each sprint. This included monitoring ticket progress, identifying blocked tasks, ensuring sprint goals were achieved, and making sure improvements from retrospectives were implemented. The Scrum Master also ensured that Sprint Planning, Daily Stand-Ups, Sprint Reviews, and Retrospectives were completed during every sprint. The team had two supervisors during the project: one main supervisor and one substitute supervisor. Meetings with the main supervisor were held every Friday, either online or in person, to present progress, discuss technical challenges, and receive feedback. The substitute supervisor supported the project when needed.

In addition to supervisor meetings, the team held internal Scrum meetings every Monday and Wednesday, usually on-site in Løkken. Each meeting included a daily stand-up where team members shared progress, discussed issues, and assigned new tasks when previous ones were completed. When in-person meetings were not possible, communication and meetings were conducted through Discord.

Several Scrum ceremonies were implemented throughout the project, including Sprint Planning, Sprint Reviews, and Sprint Retrospectives. During Sprint Planning, the team also used Planning Poker as an estimation technique to discuss tasks and esti-

mate their complexity and workload collaboratively. Jira was used as the main project management tool to manage the backlog, track sprint progress, and organize tasks. We used Jira instead of Trello because Jira supports Scrum methodologies better with integrated sprints, product and sprint backlogs, reports for each sprint, task dependencies and team collaboration in larger development projects.

Although the project followed core Scrum principles, some adjustments were made to fit the academic environment and team availability. We also used Excalidraw to visually map ideas, workflows and planning which helped team coordination. Overall, Scrum improved collaboration, accountability, and project organisation during development.

Chapter 3

Sprints

3.1 Sprint 1

Summary

Previous sprint 0 focused on setting up the tools and getting familiarized with the group and the project and as such sprint 1 was focused on building a programming foundation to work on and to have clear project requirements.

Deliverables

Product increment for sprint 1 was written high level and low level requirements, functional and non-functional requirements, use case diagram and user stories. In terms of programming, SDM, RDM, AM and main navigation layout were implemented.

Sprint Planning

The sprint goal is to implement the major modules: AM, SDM, RDM and navigation layout. From our Backlog we added all the tasks that needed to be done for our Sprint goals. Planning poker was used for sprint planning.

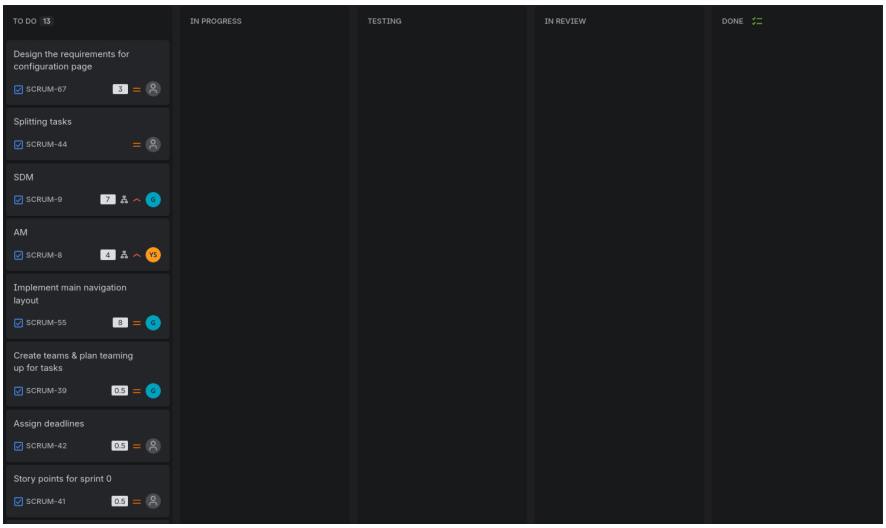


Figure 3.1: Overview of the Jira board at the start of Sprint 1

Sprint Review

When the sprint ended, almost every single task got completed, except just one task that was not completed because we found out it was not needed. Story points estimates from sprint planning were not realistic. Furthermore, code reviews took a considerable amount of time and slowed the progress more than expected.

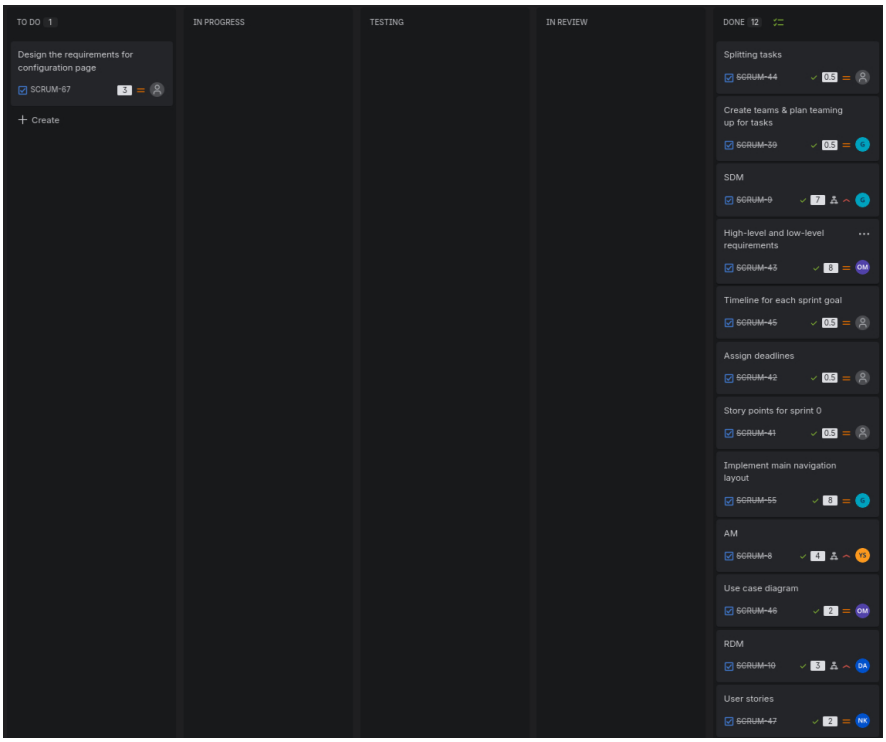


Figure 3.2: Overview of the Jira board at the end of Sprint 1

Sprint Retrospective

We did not overcommit, as we managed to deliver what was planned. Moreover, we created a solution of using an online Excalidraw canvas for decision-making to be more transparent and to encourage everyone to share their ideas. Lastly, we integrated reviews which is great for awareness and transparency, despite it taking more time.

We had issues with Jira's features and assignment of tasks. The team assigned did not show up on the board, subtasks are not easy to distribute and splitting tasks requires a lot of effort due to interdependecies, and there is no ability to assign someone to review the tasks.

Our team's story point estimate units did not match the reality, as tasks took about twice as long as expected. Therefore, there was not as much progress in the first week of the sprint. Moreover, members updated the Jira board with delay.

Distribution and assigning tasks during the Sprint Planning also came with some issues. Some tasks were assigned to people without asking them about it first. There was also some uneven distribution of tasks based on our story points, but in reality the estimates were not as accurate.

For the next sprint, we want to improve our task assigning both in person and in Jira. For the in-person assigning, we want to have more discussion on what each task is about and how it can be completed. Additionally, we will focus on assigning tasks formed as questions not statements, so that people do not feel forced to do tasks. On Jira, we will use labels to assign pairs as well as reviewers. The tasks will also have better descriptions so they are clear.

Moreover, we want to improve the progression of tasks. As tasks take longer time to complete than expected, we want to have higher transparency with why it is happening. Therefore, we will have better questions during the stand-ups:

"In fact progressing as estimated, what is in the way of the progress? Be specific."
"What was your time allocation for the task since the last meeting?"

Reflections and Conflicts

The sprint goal was reached and tasks progressed very well. As for the conflicts, for about the first week of the sprint team engagement was relatively low and team members left unheard and their opinions ignored as they were feeling pressured and

sometimes were directed to pick specific tasks. This was very prominent during time crunch situations where there was not enough time to do the tasks. In the middle of the sprint, the issue was resolved by having a shared online Excalidraw canvas during in-person meetings where tasks were listed for the session. This reduced time pressure to allocate a person for the task, team members were able to voluntarily pick tasks knowing what is needed to do and as such team engagement and working environment improved. It worked well enough that online Excalidraw canvas usage was even further expanded in the later sprints for in-person meetings.

3.2 Sprint 2

Comming...

3.3 Sprint 3

Summary

The main focus of Sprint 3 was to finish Scenario 1 for the Optimizer. Additionally, parts of backend and frontend were finalized with general connections established. For better convenience, export of the optimization results from RDM was implemented using JSON instead of CSV.

Deliverables

Delivered product increment contained Scenario 1 Optimizer implementation, frontend and backend for Production Units page and additional expansion for RDM export.

Sprint Planning

The sprint goal is to implement the connection between backend and frontend parts, implementing export from RDM. The main goal is to reach Scenario 1. The Sprint consisted of leftovers that weren't finished in the last Sprint.

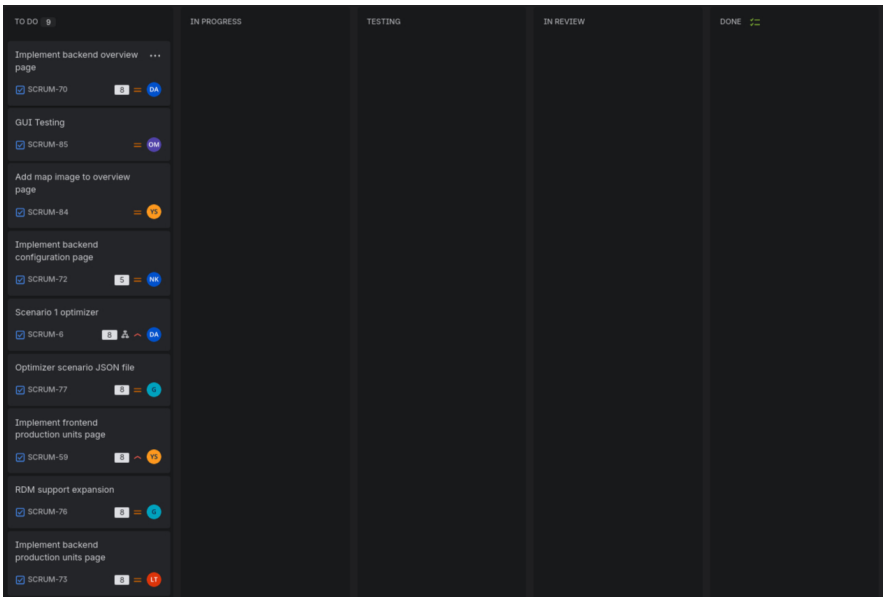


Figure 3.3: Overview of the Jira board at the start of Sprint 3

Sprint Review

The outcome of the Sprint is quite mixed. Compared to the last Sprints we didn't finish as many tasks. This could be a result of the distribution of tasks between members, some of which are not as experienced as others, as well as the over reliance of dependencies of other tasks.

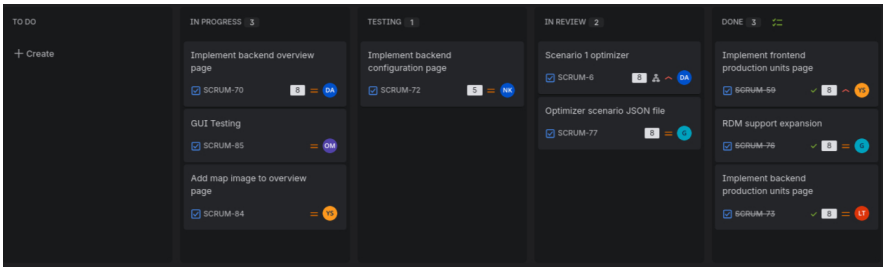


Figure 3.4: Overview of the Jira board at the end of Sprint 3

Sprint Retrospective

Reflections and Conflicts

A lot of the tasks were accumulated from the end of the previous sprint, and due to the easter break overlap with one of the meetings the sprint planning was a bit more rushed. Missed deadlines could be the result of bad task distribution and not addressing

equally each of the members capabilities due to less developed communication practices throughout previous sprints.

Although the sprint ceremonies were subject to scheduling difficulties, with more confusion as to which format to use being raised by late absence notices to in-person meetings, the results of this sprint show more transparency, communication improvements, more elaborate attempts to collaborate on resolving arising impediments.

3.4 Sprint 4

Summary

The sprint goal for this sprint was set to finalize the implementation of Scenario 2, improve the visualization of the application, and to begin working on the report.

Deliverables

The product increment during this sprint included merging the production unit page with the configuration page, finishing all backend implementation for the UI, improving the optimizer, and adding layered charts and heat grid map to the overview page.

Sprint Planning

For the Sprint Planning in Sprint 4, we decided to try a new technique for assigning tasks. Everyone pre-estimated the amount of time they expected to have for the project during the two weeks, as we expected it to make the members feel more obligated to complete their tasks. After playing Planning Poker, all members selected their tasks on Jira based on their own work estimation and the corresponding story points on the tickets. The Sprint Backlog contained a few tasks from the previous sprint that were in the final stages and were completed in between our sprints, therefore we started with tickets in the done column (see Figure 3.5).

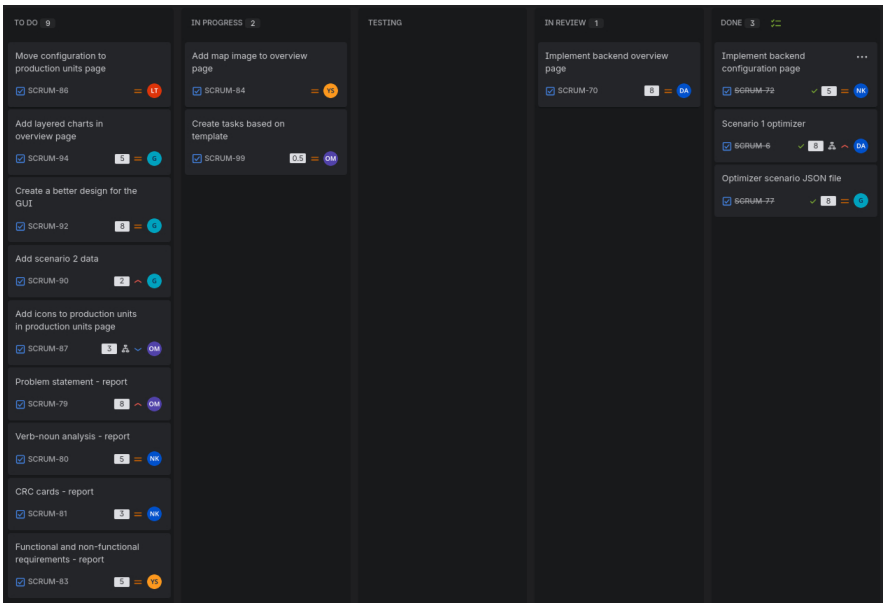


Figure 3.5: Overview of the Jira board at the start of Sprint 4

Sprint Review

At the end of the sprint, most of the tasks were completed or in review (see Figure 3.6), with two tasks in progress and only one not started. The increment of the product did not fully satisfy the sprint goal, however we progressed a lot towards finishing the application. The Sprint 4 demonstration was moved to a later date, therefore we did not receive feedback from the stakeholders at the end of the sprint.

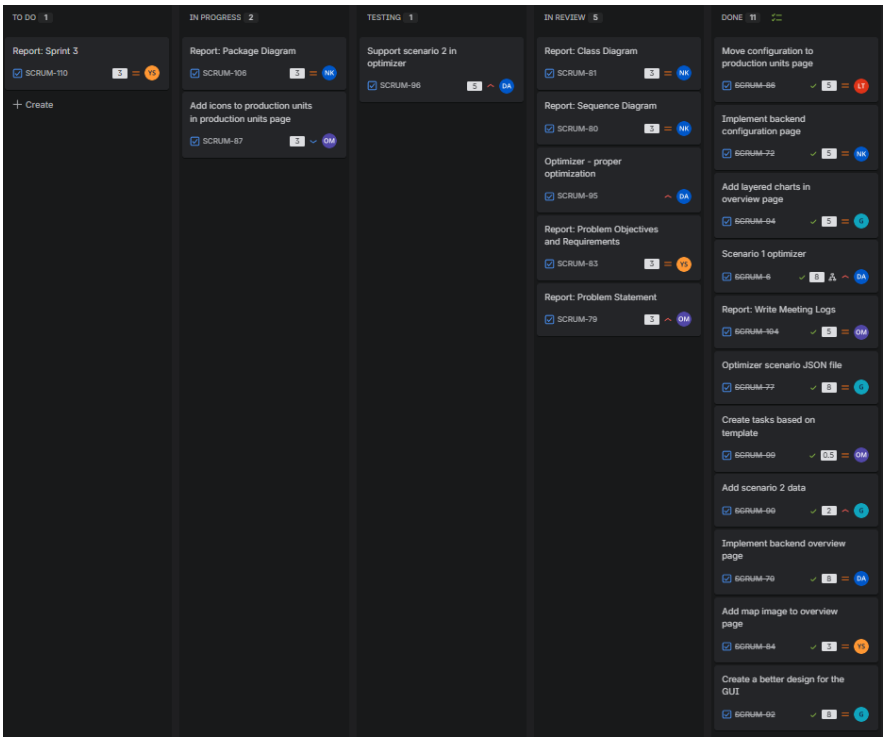


Figure 3.6: Overview of the Jira board at the end of Sprint 4

Sprint Retrospective

The Sprint Retrospective started with each member writing what worked well and what did not on our Excalidraw board, of which summary can be found in the meeting logs in the Appendix B. Everyone agreed that the tasks are progressing fairly well, and that implementation of more frequent communication over text messages is working. However, the issue of poor attendance was brought up. It was common that the meetings had only three to four members attending. Combined with people often getting late, the meetings seemed like they were optional and this created some discouragement, as the communication around the attendance was also quite poor.

Reflections and Conflicts

To resolve the issues, we first identified the reason behind team members getting late or being absent, which turned out to be an issue with oversleeping. We came up with a solution to move the time of the meeting to a later hour to see if that will improve the attendance. Moreover, during the discussion another issue was found. Some members felt that the ending of the meetings is not clear enough, and to solve it we decided to have the end more clear.

3.5 Sprint 5

Summary

During this sprint we have improved the optimizer by adding new Scenario 2 features such as support for emissions and consumptions, and handling of negative prices. The UI was also improved and the codebase was refactored to better support multiple scenarios and data handling. Additionally, we worked on the report by updating sections and handled ticket and pull request reviews to support overall project quality.

Deliverables

At the end of the sprint, we have delivered an improved optimizer with Scenario 2 features, a refined UI with better data and scenario support, and a refactored codebase supporting multiple scenarios. In addition, updated documentation and report sections, along with improved diagrams and reviewed pull requests, were completed.

Sprint Planning

The sprint goal is to work on different parts of the report, including the sprint implementation, technical architecture, abstract, Scrum implementation, problem statement, diagrams, and the problem objectives and requirements. Additionally, some coding tasks were assigned, such as GUI testing and supporting implementation-related improvements. Overall, the focus is on completing both documentation and technical components to ensure consistency and final integration of the project.

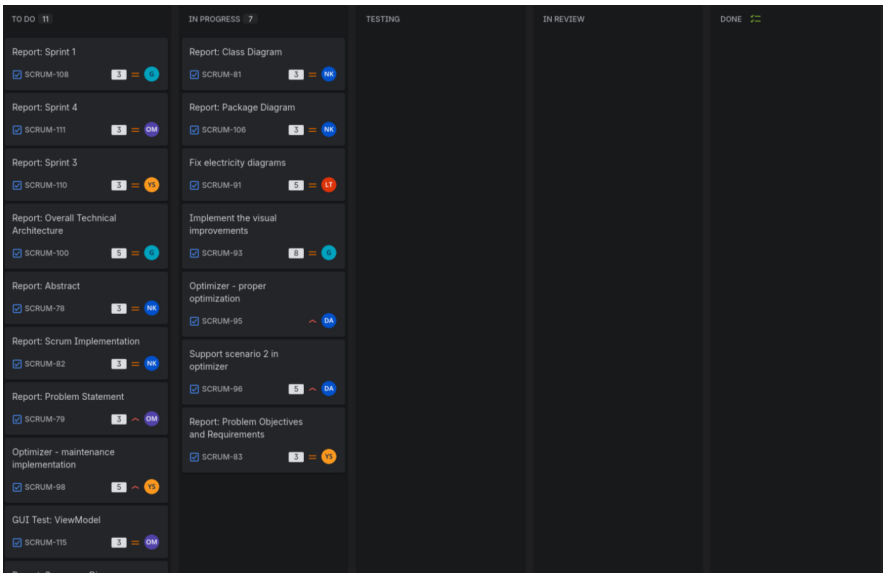


Figure 3.7: Overview of the Jira board at the start of Sprint 5

Sprint Review

The sprint ended with most of the planned tasks completed, showing solid progress from the team. However, several tickets got stuck in the review stage, which slowed down the overall workflow. In addition, code review and validation took longer than expected, creating a bottleneck that delayed the final integration of some features. Despite these challenges, the team delivered most of the intended functionality.

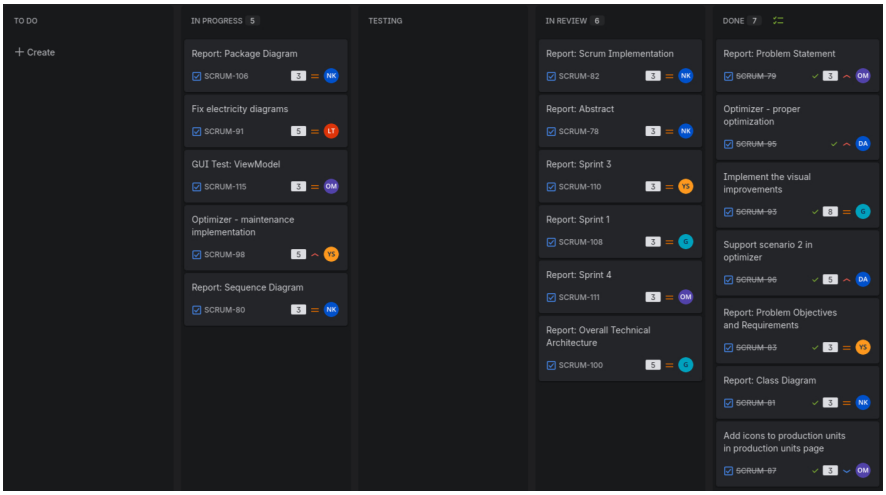


Figure 3.8: Overview of the Jira board at the end of Sprint 5

Sprint Retrospective

Communication about the tasks seems to have improved and the progression is going well. The members have started to give more elaborate answers to questions and are voicing their opinions more. Lastly, the workload was adjusted to the scheduled Math 2 exam.

The sprint progress was slower due to external factors such as the Math 2 exam, a slightly reduced availability due to AOP homework and other communal issues. Additionally, absences from meetings influenced progress. Notifications of absence were often sent very late, which caused organizational issues. Meetings were not always rescheduled when key participants could not attend. The group contract, which requires attendance and at least one-day advance notice of absence, was not consistently respected or enforced. Half of the team members did not meet the estimated workload defined during sprint planning by a significant margin. A large number of tasks accumulated in review and were repeatedly moved between "In Review" and "In Progress," causing delays. This issue was further worsened by slow review activity and insufficient follow-up. As a result some tickets from the previous sprint remained in the review cycle and were not completed during this sprint. Additionally, there were inconsistencies in expectations regarding hybrid meetings and short-notice changes, which further affected coordination and planning. Although team communication has improved overall, some members still occasionally forget to inform the group about absences or meeting arrangements, which contributed to coordination issues. During one of the meetings, attendance was split between online and on-site participants, which highlighted the lack of a consistent approach to hybrid meetings.

The attendance issue is planned to be solved by requiring team members to send a message in the group one day before the meeting, confirming whether they can attend in person at a specified time and location or, if not, whether they can attend online. This approach accounts for the possibility of late notifications about online only attendance and allows the team to prepare for hybrid meetings in advance. If a significant number of team members are unable to attend, the meeting should be rescheduled. Regarding the review process, team members will be reminded that, as previously agreed, participation in code review is required from everyone to ensure consistent workflow and progress tracking.

Reflections and Conflicts

Communication and engagement improved, and the workload was adjusted for the Math 2 exam. However, progress was slowed by attendance issues, late notifications, and tasks getting stuck in review or not being completed from the previous sprint. Coordination was also affected by inconsistent meeting arrangements and expectations. For the next sprint, we will enforce earlier attendance confirmation, improve planning for hybrid meetings, reschedule when necessary, and set clearer deadlines for reviews to improve flow and completion rate.

3.6 Sprint 6

Comming...

Chapter 4

Solution (Technical Aspect)

4.1 Package Diagram

The package diagram shows how the Heat Production Optimization system is organized by breaking the application into separate modules, each with its own specific tasks. The system is made up of different parts: the Asset Manager, the Source Data Manager, the Optimizer, the Result Data Manager, and the Data Visualization components.

The Asset Manager takes care of keeping fixed information about the heating grid and production units, while the Source Data Manager deals with changing data like heat demand and electricity prices. The Optimizer is the main part of the system. It takes information from both managers to figure out the best and most cost-effective plan for producing heat. Once the calculations are done, the results are saved in the Result Data Manager and shown to the user using the Data Visualization module.

All parts of the system are linked to a common storage area that uses CSV and JSON. This setup lets the system access input data and store the results of optimizations. This package layout was selected because it clearly distinguishes between how data is managed, the logic for optimization, and how information is presented. Because of this, the system is simpler to understand, easier to take care of, and can be updated with new features in the future. It is relevant because it explains the system's structure, the role of each module, and the data flow, helping to understand and justify the design shown in the package diagram.

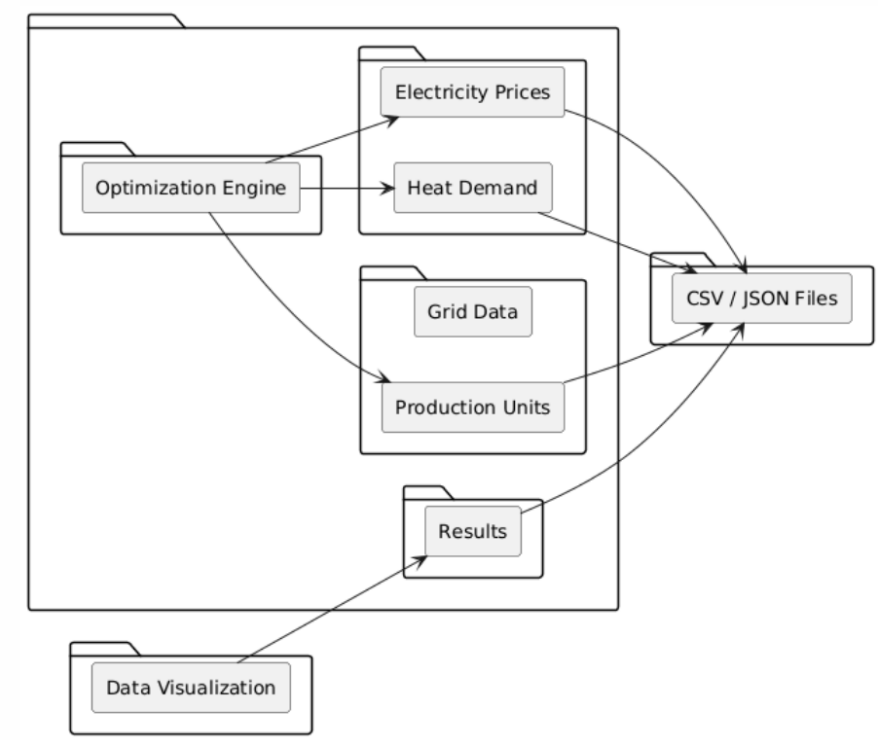


Figure 4.1: Package Diagram

4.2 Class Diagram

The diagram shows a system made up of different parts, with the Data Manager acting as the main one that brings everything together. It links the main parts - Asset Manager (AM), Source Data Manager (SDM), and Result Data Manager (RDM) - and takes care of important tasks like starting up the system, bringing in data, doing the optimization process, changing the selected time period, and sharing the results.

The AM is in charge of handling assets and scenario details. It stores a collection of Asset objects, which have details such as capacity, costs, and emissions, along with ScenarioData that sets priorities and maintenance rules. The SDM handles input data that changes over time using SourceData objects, which keep track of information like timestamps, heat demand, and electricity prices. The RDM is responsible for handling the output data (ResultData), including both summary statistics and detailed results. The data is generated by the Optimizer and can also be exported for further use.

The Optimizer uses information from AM and SDM to do calculations, like looking at costs and making plans for production. Additional parts such as ScenarioLoader and JSONPersistence help manage data by allowing the process of loading and saving information.

In short, the Data Manager connects all parts of the system together. Each manager handles a specific type of data. The Optimizer uses this data to create results, which are then viewed and shown through the ViewModels.

This design is relevant because it separates responsibilities between components, making the system easier to maintain and extend. Each manager focuses on a specific type of data, while the Data Manager coordinates the workflow. This clear structure improves scalability, simplifies updates, such as changing data sources or optimization logic, and ensures that results can be efficiently processed and reused.

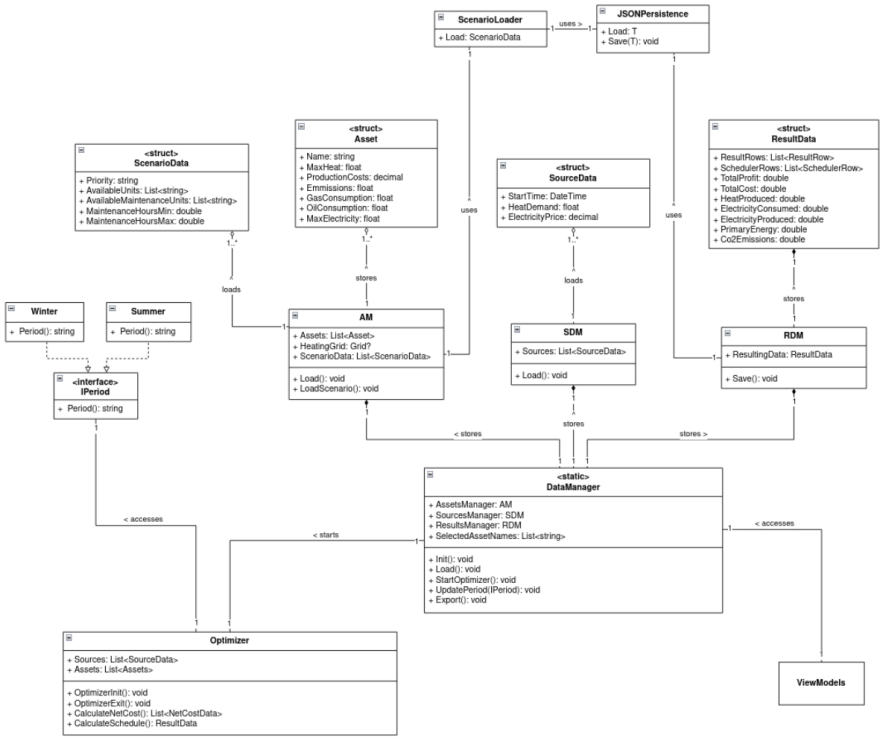


Figure 4.2: Class Diagram

4.3 Activity Diagram

The activity diagram represents the user interaction flow with the heat optimization system. The user begins with choosing the desired scenario followed by the heating period. Then, he or she can either proceed to edit production units or directly to the execution of the optimizer. After the system calculates the results, the user can export the data, view the charts or exit the program. The decision paths shown on the diagram represent the flexibility of the flow as the user can revisit earlier steps.

The diagram models the optimization execution process from the perspective of the

user, rather than low-level implementation. Therefore, it provides a clear and intuitive representation of the workflow, which can be used to communicate the system behaviour to non-technical readers or stakeholders. Moreover, the diagram is used as a validation of the business objectives and high-level requirements, and to provide documentation for future maintenance.

Actions and the flow presented on the diagram connect to the implementation of UI functions and how user actions trigger system events. The support of loading preconfigured scenarios allows the user to run the optimization right after selecting the scenario and heating period, without the need to edit the units. The optimization and production units' configuration pages are separate, and they can be accessed at any time. Once the optimization is started by pressing the Run Optimization button, the charts are updated with new data, and the export button becomes enabled.

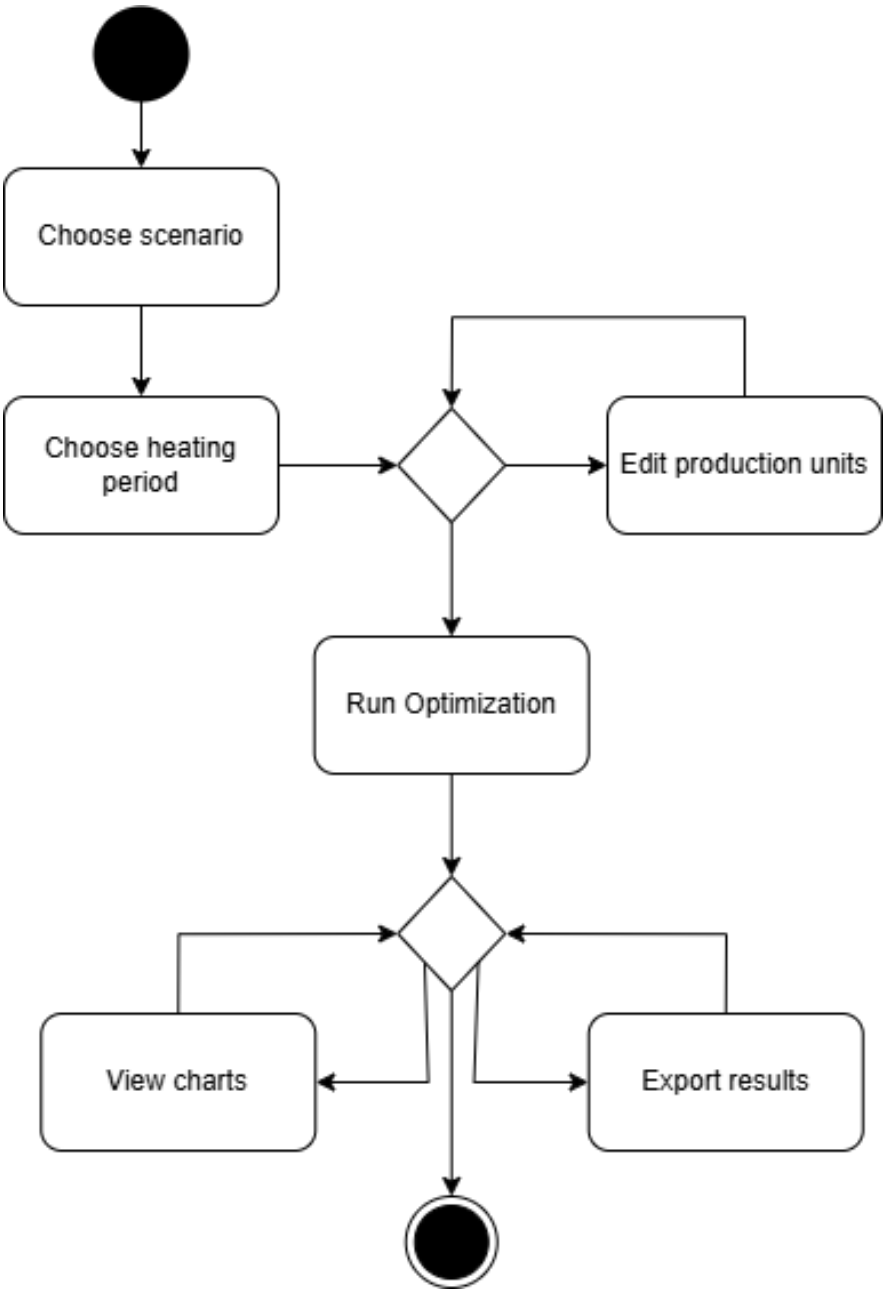


Figure 4.3: Activity Diagram

4.4 Sequence Diagram

In this case, the process starts when the user sets up a scenario and chooses the right assets using the ViewModels, which then starts the system setup through the Data Manager. The Data Manager gets the necessary information by getting asset details from the Asset Manager and gathering time-related data, like demand and price, from the Source Data Manager. Once all the needed data is ready, the optimization process starts, and the prepared inputs are sent to the Optimizer.

The Optimizer goes through each time step one by one, getting the needed demand and price information. For each asset, it figures out the costs needed to produce it, sorts the assets by how efficient they are economically, and finds the best way to assign production while following the rules of how the operations work. Once the optimization is done, the results are collected and saved by the Result Data Manager. When the user asks for them, the results are sent back to the ViewModels, and then they are shown using things like charts and key performance indicators. This design was chosen to separate data management, optimization and presentation into distinct components. This is relevant because it creates a clear workflow, improves maintainability, and makes it easier to handle complex optimization results efficiently.

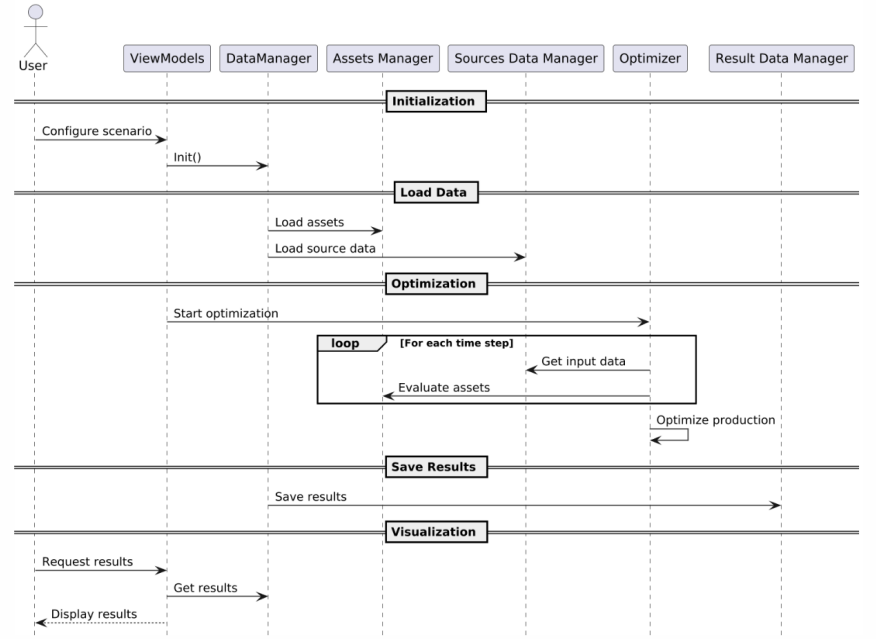


Figure 4.4: Sequence Diagram

4.5 Overall Technical Architecture

Technology Stack

Our technology stack for 2nd semester project was C# for all frontend and backend, C# Avalonia framework for frontend with data bindings, LiveCharts for data visualization, and NUnit for testing. Working with Avalonia proved to be more challenging than expected as online resources, documentation were not as high-quality and satisfactory as expected. Provided abstractions by Avalonia and their suggested design patterns did not allow for scaling code well while keeping codebase clean - data duplication,

minor hacks for reactivity and passing data were required to achieve desired result and many of their official examples introduced substantial complexity. Additionally, LiveCharts library was limiting because it had issues with crashing outside of C# code. For loading, storing and exporting state, JSON files were used. Due to small project scope, low volume of data, high flexibility, easier data management and the project being a standalone desktop application, storing data in JSON was more fit for the task than using traditional relational databases such as PostgreSQL or MySQL. We have considered using Docker to standardize our development environment even further but we have rejected it because there were no major issues of the application not working as expected on a different computer. There was only one issue during sprint 1 related to different path handling on Windows and Linux but it was fixed quickly. Furthermore, we have considered integrating CI to our GitHub for running tests automatically but it was ruled out as unnecessary because during our code reviews we already run tests among other review tasks and there would be no increase in development velocity from such integration at such project size.

Seperation of Concerns

To keep our codebase clean, make it modular, easier to work with, and collaborate, we used 3-layer programming - application was split to presentation, domain and data layers. Additionally, the application was segmented into 5 modules. Asset Manager (AM) responsible for loading static data, Source Data Manager (SDM) responsible for loading dynamic data, Result Data Manager (RDM) for storing result data and exporting. They belonged to the data layer. Optimizer (OPT) responsible for considering all available data, scheduling available units optimally to reduce costs and scheduling units for maintenance, and Data Manager (DM) which connects the data layer to be accessible to the Optimizer module and the presentation layer. Data Manager acts as a thin wrapper around the data layer for the Data Visualization module to access it and for Optimizer it provides wrapper methods to call. These 2 modules belong to the domain layer. The Data Visualization (DV) module forms the entire part of the presentation layer. It is the single largest module and it is further broken down to different views and respective view models. Different tab groups are different views such as vertical scenario tab group and upper page tab group. Each different page tab is a different view, such as overview, optimizer and production units page. Smaller elements such as different graphs and edit production unit popup have their own views. Splitting data visualization to many different views proved helpful for readability, reuse and reducing merge conflicts.

Architecture and Rationale

We implemented a 3-layer programming architectural pattern as our foundation for the codebase. In the Data Visualization module, each view had their respective view model with data bindings and commands, and it followed MVVM design pattern. We have chosen MVVM over other GUI architectural patterns because it increases readability, reduces boilerplate, and reduces code complexity as long as we stay to simpler Avalonia features. To keep GUI reactive to data changes, we have used the observer pattern. Whenever data updates, a message is sent notifying that refresh is required and GUI components subscribe to the channel. Observer pattern looked like a perfect candidate for the task because it is heavily used in GUI development, is tried and tested, and keeps single responsibility principle with no significant downsides. However, due to the difficulty of passing events to components, only some parts of the code use observer pattern. Data Manager module which connects different modules acts as a singleton by using C# static members.

For the most part, our code follows DRY principle of not repeating code by using modules and utility classes. The benefit of improved readability was a decisive choice. However, some Avalonia components were proven to be difficult to cleanly refactor to proper modular components without adding significant complexity. Furthermore, the codebase follows YAGNI principle of not having unnecessary code which is unused. Quite a few parts of our codebase were refactored and removed during the development which were not needed anymore or a better, more simple approach was found. For the most part, SOLID principles were partially adhered to. Single responsibility is used heavily by splitting many features into different modules and classes. For Object-Oriented Programming class design, composition over inheritance was heavily used. Various edge cases of inheritance such as diamond problem and potential complex situations later down the line deterred us from using inheritance and composition had no drawbacks in our case. None of the classes besides the Data Visualization module use inheritance which only inherit Avalonia base classes to implement functionality. As such, Liskov substitution principle is upheld. Open-closed principle is partially followed, as due to heavy use of composition, changing related classes can extend the behavior of the classes without modification but there are no specific mechanisms beyond composition to allow extending class functionality without modification. Interface segregation principle and dependency inversion principle were not strictly adhered to because there is only one interface in the codebase. Instead, composition on concrete classes were used. We have decided to follow KISS principle to keep simplicity instead of introducing additional abstraction layers using interfaces. In such a relatively small project, additional simplicity and reduced boilerplate allowed for quicker iteration.

Chapter 5

Implementation and Technical Realization

This chapter explains how the system was technically designed, implemented, and improved during the project. Its purpose is not only to show what was built, but also how the group worked technically and why specific engineering choices were made.

5.1 Frontend Implementation

Frontend for this project was implemented in C# with Avalonia UI framework and LiveCharts for graph visualization. MVVM pattern and data bindings were used with Avalonia. Frontend is split into supporting elements and pages.

Supporting elements include sidebar navigation panel, top navigation tab bar and chart component. Sidebar navigation panel has the logo of our project and a list of scenarios which can be switched by clicking on buttons. Top navigation tab bar contains tab names for each page and a combobox for selecting different periods. The period checkbox is separate for each scenario. The chart component is used for dynamically adding charts to the page. It is used in the optimizer page.

There are 3 pages - overview, optimizer and production units page. After our initial Figma prototype, we had 4 pages, the fourth one being the configuration page which handled scenario specific data. However, later down the line we have merged the production units page and configuration page into one production units page because it improved user experience, clarity and reduced clutter. The first page, overview page, contains a map of the Heatington city and various general graphs such as heat demand

and production, electricity prices which are populated when the optimizer is run. The second page, optimizer page, has a button to start the optimizer and to export the results of the optimization. Furthermore, after the optimization completion, it shows totals for costs, revenue, amount of heat produced and consumed, energy used and amount of CO2 emissions. Maintenance periods are shown. The graphs with detailed optimizer information such as unit scheduling, price for 1 MWh for each unit at a given hour and costs for each unit at the given hour are shown. It is possible to use a combobox for selecting specific units to be shown in the graphs. In the third production units page, all of the production units are shown with their properties - image, name, heat production, electricity, minimum and maximum maintenance hours, can the unit be maintained, unit colour in the graphs and so on. In the same page, production units can be selected or deselected for the scenario, units can be edited, deleted or new production units can be added.

Each component and page have their respective view models responsible for filling the data required for the component. All of the data and actions performed go through Data Manager directly or indirectly. Much of the data displayed in the UI is displayed directly without any modification and a small part of data shown is heavily processed data with inputs coming from Data Manager. Such data include data provided to the charts which contains numerous LINQ queries, handling unit selection, dynamic colors and empty data periods.

During our frontend implementation, we have faced major challenges writing clean code with Avalonia UI framework. First of all, data which is stored outside of Avalonia is difficult to update reactively without desynchronization issues. For example, Data Manager provides result data for the optimization and the result is stored outside of Avalonia code because it is part of the domain layer. Avalonia does not update the UI with new data automatically. Instead, C# events could be passed around through the entire hierarchy of UI components to the Optimizer view model to call a refresh method once new data is available. However, such "prop drilling" to pass a refresh event was proven difficult as getting data to the view model from other components would have required changing every view and view model along the UI hierarchy. Sadly, some less efficient reactivity methods needed to be used, such as calling refresh manually from the view model itself when it thinks new data should arrive. For instance, after calling the run optimizer method, a refresh method is called for the Optimizer view model to populate fresh data. Such a method helped to solve the issue but due to its nature of not being so reactive to changes, many bugs have arisen where data was stale and caused desync. Adding more points to refresh data helped a bit more. In some cases, Data Manager was used to store data meant to be accessed between multiple components - such as current scenario and period. It worked but it caused additional

issues with multiple scenarios and periods and required granular tweaks to make it be more reliable. In hindsight, paying the cost of changing many UI classes to pass around data to components could have been better and more reliable than relying more on Data Manager for passing data around between components. Furthermore, a clear and consistent way to pass data around Avalonia components from the project start could have resulted in better results. Even multiple attempts to refactor code were not able to resolve major issues of the codebase.

In addition to Avalonia C# framework, LiveCharts library had its own issues too. After adding a few graphs with a total of around 3 thousand points, there were visible performance issues when resizing the application window. Because view model data did not change during application window resizing, it could have only been caused by the LiveCharts library. In the Optimizer page where there were more graphs than could fit in the screen, charts which were rendered off screen were hidden or distorted even when they scrolled into view. The hover tooltips would still show up but the visible chart had issues. The only workaround we found was resizing the window to refresh LiveCharts charts views. Furthermore, a LiveCharts feature of adding disjointed line sections was used. For example, when a unit is running, then is powered off and turned on again, the points should not be connected and the gap should be shown instead. However, the lines at the ends of intervals in the charts had higher values and caused confusion about what values are really at those endpoints. But the points were rendered correctly and the hover tooltip was showing correct values which could only mean there is a bug in the LiveCharts library itself.

Frontend was designed to be a balance between being user-friendly and having in-depth features. It caters to simple users by being graphically appealing, having simple summary statistics and summary graphs, limiting the number of pages visible and providing an intuitive interface. For advanced users, we have added a feature to create, edit or delete production units and to change scenario data dynamically. Furthermore, they can inspect the optimizer better with the help of detailed scenario graphs which show granular key metrics such as unit cost for 1 MWh, unit cost for unit and scheduling of each unit. We had two major Figma design iterations over the course of the project and a few of the changes were made directly in code after feedback from the stakeholders. Sprint reviews helped with improving the frontend incrementally by collecting stakeholder feedback at regular intervals and changing user interface based on the feedback.

5.2 Backend Implementation

Backend is composed from the Optimizer and Data Manager. Optimizer is responsible for calculating optimal cost for the provided time period and assets and Data Manager is responsible for exposing data layer to domain and presentation layers.

Optimizer is a central part of the backend. Data Manager configures optimizer by loading sources, assets and any scenario specific data. During Optimizer initialization, data sources and assets are validated. After validation, net cost for each unit at each hour is calculated. Net cost is the cost of 1 MWh of produced heat for a given unit at a specific hour. Net cost calculation takes into account production costs of the unit and any electricity produced or consumed by the unit during heat production. After initialization, the optimizer is started in a separate thread. Optimizer tries to find an optimal maintenance period for 1 production unit from the list of available units to be maintained. Optimizer iterates over all possible maintenance periods for each unit and selects the maintenance period with the lowest cost. For each possible maintenance period, the Optimizer step is run which calculates the cost, heat production, electricity consumption among other properties with the given maintenance period. During this step, maintenance period for the unit is applied and for each hour the units with the lowest cost are selected until the heat demand for an hour is filled. After heat demand for every hour is fulfilled, the result with granular detailed data and total summaries are returned back to the maintenance period iterator. The lowest cost maintenance period with its result data is stored in RDM. The main thread is notified about the completion of the optimization.

Currently, Data Manager belongs in the domain layer and it is responsible for exposing AM, SDM and RDM to optimizer and frontend. Data Manager does this by providing a thin wrapper over AM, SDM, RDM and by providing methods which abstract interacting with the optimizer and updating scenarios and periods. Data Manager implements singleton pattern using C# static members because there is only one globally accessible instance of Data Manager.

5.3 Data Management

Explain how data was modelled and managed. Cover:

- Data structure.

- Data validation and consistency considerations.
- How the data structure integrates with the backend (if relevant).

Chapter 6

Discussion & Reflection

Reflect on and discuss the software engineering practices and processes adopted this semester. Describe ideas for future work that could improve the project. Include only practices and techniques actually adopted in your project. Delete any subsection that does not apply.

6.1 Scrum

Reflect on the extent to which Scrum practices improved your project work. What outcomes and behaviours — e.g. better code quality, better accountability — changed? What challenges were experienced?

6.2 Requirements Engineering

How did the practices you adopted elevate the quality of your work? Which ones will you retain and improve in future projects, and why?

6.3 Refactoring

How did refactoring help your project's code quality, skills balance in the group, etc.? (Delete this section if refactoring was not used.)

6.4 Code Review

How did code review help with code quality and knowledge sharing? What were the challenges and how did you overcome them?

6.5 Testing

To what extent did testing help the quality of your product? What were the disadvantages — e.g. slow productivity, intellectually unstimulating — and how do you plan to adopt testing in future projects?

6.6 Overall Group Reflection

Include an overall reflection on the project outcome:

- Whether all requirements were met or some were changed (e.g. a comparison table).
- Whether the product solves the stated issue.
- Whether the product leads to unforeseen problems that would be addressed with future development.
- Possible future development in terms of features, scalability, or other important factors.

Chapter 7

Conclusion

This chapter directly answers your project requirement. Structure it as follows:

- Briefly summarise the key findings.
- Provide a clear and final answer to the problem statement.
- Highlight any objectives that were not fully achieved.
- Suggest possible directions for future work.

References

List all external sources, libraries, and documentation used to support the research, design, and implementation of your project. Include a link to each resource. Use a consistent citation style throughout the report.

AI Usage

This section must document the use of Artificial Intelligence (AI) tools in the project. If AI tools were used at any stage — writing, design, coding, research, brainstorming, or data analysis — their use must be clearly described and justified. Describe:

- Which AI tools were used (e.g. ChatGPT, Copilot).
- How the tools were used (e.g. brainstorming, improving language, summarising information).
- Which parts of the project were influenced by AI, if any.
- How the AI-generated content was reviewed or modified by the student.

Students remain fully responsible for the final content of the report, including the correctness of information, code, and analysis. AI tools must only be used as support and not as a replacement for independent academic work. **You must not use AI for:**

- Code generation.
- Software design decisions.
- Test use cases and related artefacts.
- Product Backlog and requirements engineering artefacts.
- Writing or enhancing report text (feedback and proofreading are permitted).
- UI generation.

The use of AI must follow the official guidelines provided by the University of Southern Denmark. Ensure your use of AI complies with the rules described at:
https://mitsdu.dk/en/mit_studie/kandidat/negot_kandidat/vejledning-og-support/aipaasdu

Appendix A

Team Contributions

Table of precise contributions of each team member for both the report and the development part.

Student	**Report Contributions**	**Development Contributions**
David Avramov		
Gintaras Zulys		
Leon Jürgen Thye		
Nadia Kristensen		
Oliwia Roksana Moskalik		
Yaroslav Savenko		

Table A.1: Team Contribution Overview

Appendix B

Meeting Logs

Include all meeting logs here: Scrum planning, stand-up meetings, Scrum review, and retrospective logs that were not included in the main body.

Appendix C

Contracts

Supervisor Contract

SDU 2nd Semester Project – Supervisor-Student Collaboration Contract

Department/Programme:	Software Engineering
Semester/Year:	Spring 2026
Project Title:	Heat Production Optimization
Supervisor:	Alan Sieradzki
Student(s):	Oliwia Roksana Moskalik Nadia Kristensen Leon Jürgen Thye Gintaras Zulys David Avramov Yaroslav Savenko
Project Period:	January – May 2026

ECTS: ☐ 10 ECTS

1. Purpose

This contract specifies the collaboration between students and their corresponding supervisor. The agreement sets the rules and expectations from both parties. Following this contract ensures good communication and collaboration throughout the semester project development

2. Expected Student Workload

Each student, to achieve the goal of 10 ECTS points, is expected to commit:

1 ECTS = 27,5 h;

10 ECTS = 10 * 27,5h = 275h

275h / 12 weeks ≈ 23h/week

23 h/week – 4 h/week = 19 h/week

In total 19 h/week for the development of the semester project.

3. Roles and Responsibilities

3.1. Students responsibilities:

- Communicating about any unresolved issues;
- Preparing materials specified for the given deadline;
- Meeting deadlines given for any submission task;
- Working together in a collaborative way with other students in the group;

- Ensuring that everyone is aware of all the modifications, and that the project stays at the similar level of expertise for each student;
- Helping each other if anyone is falling behind;
- Documenting their progress in an Agile environment;
- Writing the report, combining all of the progress and including the given template;
- Maintaining academic honesty and correct referencing;

3.2. Supervisor responsibilities:

- Ensuring good progress of the group;
- Giving a satisfactory answer to any questions students may have in regards to the project;
- Notifying the group about any upcoming deadlines and any given tasks;
- Giving feedback about the progress and the development process, so that the students meet the initial requirements;
- Clearly stating objectives and requirements of the course;
- Supporting the group in case of any issues that might arise and clarifying them the coordinator of the course;

Note: The supervisor is not responsible for any technical related work, writing the report or completing tasks on behalf of the students.

4. Communication

Communication between the parties is expected to be conducted in a respectful manner, ensuring the well-being of all participants.

In the event that any questions arise, students are welcome to contact the supervisor. The supervisor is required to respond within at least two working days after the question has been submitted. For example, if a question is submitted on Friday, the supervisor's response is expected by Tuesday.

The form of communication between the parties must be agreed upon in advance, whether via email or another communication channel.

The same applies to the meeting format: the student and the supervisor must agree on one of the following methods: an online meeting conducted via an agreed-upon communication platform or an in-person meeting at a mutually agreed location.

5. Meetings and Attendance

Both parties are expected to attend student–supervisor meetings at the agreed-upon time. The date and time of each meeting shall be determined mutually by the student group and the supervisor. Meetings should be held on average at least once per week for at most one hour. The format of the meetings shall also be agreed upon by both parties.

Students are expected to attend progress meetings every other week to present updates on their development work. These meetings are scheduled on Fridays from 10:15 to 12:00.

In addition, students are required to attend all mandatory meetings according to the schedule provided on itslearning.

In the event that a party is unable to attend any of the meetings specified above, the other party must be notified in advance, at least one day before.

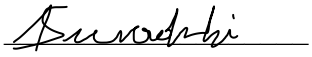
6. Report Writing

The students are responsible for conducting the documentation of their project and writing a report summarizing their development process. The template for the report is given on itslearning platform, where there is a provided description for each mandatory chapter. The template serves as a reference and does not restrict students from adding additional chapters, provided that all specified requirements are met. The encouraged format used for writing the report is LaTeX, which can be created in a shared Overleaf project. However, other formats are acceptable, as long as it fulfills SDU restrictions about an academic report.

The writing phase for the report should start no later than the 20th of April.

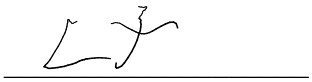
7. Agreement

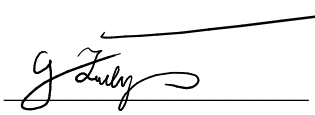
By signing the agreement both parties agree to the given terms, and agree on the following agreed upon rules.

Supervisor name Alan Sieradzki Signature:  Date: 23/02/2026

Student name Oliwia Roksana Moskalik Signature:  Date: 23/02/2026

Student name Nadia Kristensen Signature:  Date: 23/02/2026

Student name Leon Jürgen Thye Signature:  Date: 23/02/2026

Student name Gintaras Zulys Signature:  Date: 23/02/202

Student name David Avramov Signature:  Date: 23.02.2026

Student name Yaroslav Savenko Signature:  Date: 23.02.2026

Team Contract

SDU 2nd Semester Project
Group Contract

Department/Programme:	Software Engineering
Semester/Year:	Spring 2026
Project Title:	Heat Production Optimization
Student(s):	David Avramov Gintaras Zulys Leon Jürgen Thye Nadia Kristensen Oliwia Roksana Moskalik Yaroslav Savenko
Project Period:	February – May 2026

1. Purpose

This contract specifies the collaboration between team members. The agreement sets the rules and expectations of each member. Following this contract ensures good communication and collaboration throughout the semester project development.

3. Responsibilities

Each member of the group is responsible for:

- Communicating about any issues;
- Meeting deadlines given for any task;
- Working together in a collaborative way with others in the group;
- Replying to messages in 2 working days;
- Maintaining academic honesty and correct referencing;
- Informing the rest of the group if they cannot attend a meeting;
- Informing the rest of the group if they will be late for a meeting;

If responsibilities are not met and no improvement will be made after discussion with the group, the problem will be raised to the supervisor.

4. Communication

Communication between the group members is expected to be conducted in a respectful manner, ensuring the well-being of all participants. Bringing up any difficulties and problems within the project and the group should be encouraged and met with friendliness.

5. Meetings and Attendance

All members of the group are expected to attend all meetings. Two meetings are scheduled every week, on Monday at 10:00 and on Wednesday at 08:15. The duration of the meetings should not exceed two hours. The meetings are primarily in-person, however it is possible to attend the meetings online if necessary.

The meetings are scheduled around SCRUM ceremonies. First Monday of a sprint is the *sprint planning*, and last Friday of a sprint during Software Engineering lecture is the *sprint review* and during supervisor meeting is the *sprint retrospective*.

In the event that a member is unable to attend any of the meetings specified above, the other members must be notified in advance, at least one day before. In case of arriving late to a meeting, the rest of the group should be notified before the start of the meeting. If the group is not notified, the absent or late person is responsible for bringing snacks to the next meeting.

7. Agreement

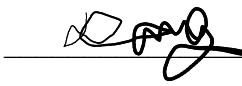
By signing the agreement each member agrees to the given terms and on the following agreed upon rules.

Student name Oliwia Roksana Moskalik Signature:  Date: 02/03/2026

Student name Gintaras Lulys Signature:  Date: 02/03/2026

Student name Nadia Kristensen Signature:  Date: 02/03/2026

Student name Leon Jürgen Thye Signature:  Date: 02/03/2026

Student name David Avramov Signature:  Date: 02/03/2026

Student name Yaroslav Savenko Signature:  Date: 02/03/26

Appendix D

Code Authorship and Review

Information about who is the author and reviewer for each part of the code, e.g. classes and components.

Appendix E

Table of Requirements

Functional Requirements

Non-Functional Requirements